

Chicago Taxi Demand Analysis

Advanced Analytics and Applications

Team 03



Authors:

Angela Feng (7397232)

Moritz Lang (7430881)

Beate Kranz (7435471)

Lennart Jekel (7444132)

Contact: ljekel1@smail.uni-koeln.de

Supervisor: Univ.-Prof. Dr. Wolfgang Ketter

Co-Supervisor: Janik Muires

Chair of Information Systems for Sustainable Society
Faculty of Management, Economics and Social Sciences
University of Cologne

2026-07-26

Table of contents

1	Problem Description	1
2	Data Description	1
2.1	Data Sources and Scope	1
2.2	Spatial Coverage and Limitations	1
3	Data Preparation	2
3.1	Cleaning and Spatial Harmonization	2
3.2	Complete Panels and Predictive Split	2
4	Data Analytics	2
4.1	Descriptive Spatio-Temporal Analytics	2
4.2	GMM Hot-Spot Analysis	3
4.3	NN	4
4.4	Support Vector Machine	5
4.4.1	Results	6
4.4.2	SVC	6
4.4.3	Further Improvement	6
4.5	NN compared to SVM	7
5	Smart Charging Reinforcement Learning	8
5.1	Motivation and Setup	8
5.2	Results	9
6	Conclusion	10
7	Appendix	11
7.1	Supplementary Data and Preparation Material	11
7.2	Supplementary GMM Diagnostics	14
7.3	Neural-Network Details	17
7.4	Neural-network baseline training configuration	18
7.5	Neural-network baseline validation results	18
7.6	Trivial baseline test performance	19
7.7	Neural-network baseline diagnostics	19
7.8	Final census-tract 1h test scores	19
7.9	Final model test MAE by spatial unit	20
7.10	Final NN and SVM pipeline comparison	20
7.11	Neural-network grid search	21
7.12	Grid-search winner training history	22
7.13	Appendix SVM	22
7.13.1	Bug found in cuml	22
7.13.2	Steps in developing SVM	22
7.13.3	Pipeline SVR/SVC	22
7.13.4	Current Result SVC	25
7.14	Appendix: Reinforcement Learning	25
7.14.1	Cost Coefficient Calibration	25
7.14.2	Hyperparameters	25
7.14.3	Training Convergence	26
7.14.4	Learned Policy Heatmap	26
7.14.5	Cost Distribution	26

List of Figures

1	Temporal pickup-demand patterns.	3
2	Community Area demand and selected GMM density.	4
3	Test-metric comparison of SVR and the three baseline models.	6
4	SVR metrics across spatial and temporal configurations.	6
5	Test MAE skill scores of NN and SVM models against both baselines.	7
6	Comparison of charging policies: Q-Learning vs DQN vs Baseline.	9
9	Distributions of trip distance, duration, fare and total payment in a fixed 150,000-trip reservoir sample of the complete cleaned population. Values above the sample p99 are omitted from each panel for readability.	12
10	Hourly mean and median of retained gaps between consecutive observed trips of the same anonymized taxi.	12
7	Pickup and drop-off demand by Community Area.	13
8	Demand sparsity by spatial and temporal resolution.	13
11	GMM candidate comparison across covariance structures and K=2–30. The selected tied-covariance model with K=17 is the best BIC candidate that satisfies the component-spread and minimum-weight safeguards.	14
12	Component means and 95% probability ellipses of the selected tied-covariance GMM. Ellipses are contours rather than hard assignment boundaries.	15
13	GMM density surfaces for four six-hour windows using the selected spatial model structure.	16
14	Validation diagnostics for the Census Tracts 1h NN baseline.	19
15	Test MAE of final 1h NN models by spatial unit.	20
16	TEST comparison of the final NN and SVM pipelines.	23
17	NN and SVM test MAE by demand bucket for Census Tracts 1h.	24
18	Training history of the selected NN grid search model.	24
19	Statistics of SVC	25
20	Learned Q-Learning charging schedule and battery trajectory.	26
21	DQN charging schedule and battery trajectory.	26
22	Q-Learning training progress (rolling average reward).	27
23	DQN charging schedule and battery trajectory.	27
24	Optimal charging policy across all states.	28
25	Charging cost distribution under the learned policy.	28

1 Problem Description

For operators of ride-hailing services tactical and strategic capabilities in operations management are a precondition for market success. On the one hand, the operator needs to understand the dynamics of the market to make guided strategic decisions, like in which cities and areas a ride-hailing business is likely profitable. On the other hand, it is also crucial for the operator to be able to make daily tactical decision, for example, to predict where and when demand will be high. We have conducted an analysis of the taxi demand in Chicago, US to enable our customer to build the tactical and strategic capabilities. With a total of 13,386,706 started trips across 851 days, and 15,731 average daily trip starts, we believe that the scope of our taxi demand analysis in Chicago is a good enough proxy for ride-hailing operations. The tactical and strategic capabilities involve business and data understanding, whereby data understanding can be seen as the foundation. Therefore, the data mining goal is to describe the dynamics of the taxi market in Chicago and develop prediction models to forecast demand. Furthermore, we evaluate whether these models are sufficient to make demand forecasting for daily operations. The business goal is to argue whether a market entry in Chicago looks promising to our customer. Within this report we present the findings of our analysis.

2 Data Description

2.1 Data Sources and Scope

The analysis combines Chicago taxi trips from 1 January 2024 to 30 April 2026 with spatial boundaries, weather, calendar information and points of interest (City of Chicago, 2013, 2024, 2025). The predictive models use the more recent period from 1 January 2025 onwards. Hourly weather observations come from Midway, O'Hare and Lansing Municipal Airport, and each spatial unit is linked to its nearest station (Iowa Environmental Mesonet, n.d.). OpenStreetMap features are obtained through OSMnx and grouped into food and drink venues, train stations, shops and landmarks (OpenStreetMap Contributors, n.d.). Illinois public holidays complete the temporal context (python-holidays Contributors, n.d.). A summary of the data sources, providers, coverage and analytical roles is provided in Table 2 in the Appendix. In the cleaned taxi data, one row represents one trip; in the model-ready data, one row represents a spatial unit during a time interval. The target, `trip_count`, counts trip starts and includes genuine zero-demand observations. Predictors describe information available before a pickup occurs, including location, time, weather, holidays and nearby points of interest. Distance, fare and drop-off information are excluded from prediction because they are only known after a trip has started.

2.2 Spatial Coverage and Limitations

We compare 77 Community Areas, 878 canonical Census Tracts, 163 H3 resolution-7 cells and 976 H3 resolution-8 cells at 1-hour, 4-hour and 24-hour intervals (Uber Technologies, n.d.) (Table 4). General descriptive results use all 13,386,706 cleaned trips with a valid pickup Community Area. Cross-resolution comparisons and the GMM use a common subset of 6,049,435 trips that can also be assigned to a canonical Census Tract. This common population ensures that differences between spatial systems result from aggregation rather than trip coverage. The public data contain privacy-protected area centroids rather than exact pickup coordinates. H3 assignment changes the aggregation geometry but cannot recover street-level precision. Missing Census identifiers are also not evenly distributed across Chicago, so the filtered population may have different local demand shares. Timestamps are rounded to 15-minute intervals, taxi IDs

are anonymized, and routes and driver shifts are unavailable. These restrictions limit street-level conclusions and make inter-trip gaps only an approximation of idle time (City of Chicago, 2024).

3 Data Preparation

3.1 Cleaning and Spatial Harmonization

Preparation follows a Bronze-Silver-Gold structure. Bronze preserves the source data as Parquet, Silver applies common quality and spatial rules, and Gold creates the complete panels used for modelling. Taxi records must contain an anonymized taxi ID, a start time, a pickup centroid and a pickup Community Area. Duplicate taxi/start/location combinations and implausible distance, duration or payment values are removed; the thresholds are reported in Table 3. Drop-off information is not required because pickup demand is the target. Weather observations are reduced to one quality-controlled record per station and hour and converted into temperature, precipitation, wind and condition features. Remaining gaps are interpolated within a station. Spatial geometries are repaired, and points of interest are counted for each spatial system. Since source Census IDs and the selected boundaries can refer to different vintages, unique pickup centroids are spatially joined to the canonical Census polygons (City of Chicago, 2013). Trips without a valid reconstructed assignment remain available for the citywide Community Area description but are excluded from the common cross-resolution population.

3.2 Complete Panels and Predictive Split

For each spatial system and each 1-hour, 4-hour and 24-hour interval, all possible spatial-time combinations are generated before observed pickup counts are joined. Missing counts are set to zero. The resulting twelve panels include cyclical time encodings, holidays, nearest-station weather and point-of-interest counts. Zero-demand rows are retained because they represent real operating conditions. Lagged demand and trip outcomes are excluded: the task estimates expected demand for a location and context rather than forecasting interval $t + 1$ from interval t . The 485 complete model-period days are assigned randomly, with seed 42, to 339 training, 73 validation and 73 test days. Complete calendar days are grouped, so one day never spans several partitions, while randomization avoids a split defined by one particular season. Since the models use no lagged demand and are not next-step forecasts, later training dates do not expose validation or test targets. Preprocessing and models are fitted on TRAIN, VAL supports model selection, and TEST remains untouched until the final comparison. All twelve panels use the same date assignment.

4 Data Analytics

4.1 Descriptive Spatio-Temporal Analytics

The typical trip is shorter and cheaper than its mean suggests (Table 5). Median distance is 3.91 miles, median duration is 16.05 minutes and median fare is USD 15.25, compared with means of 7.15 miles, 21.06 minutes and USD 22.07. Long airport and cross-city trips create these right-skewed distributions. Demand is lowest around 03:00 and rises sharply after 06:00 (Figure 1). It reaches about 1,156 pickups per date-hour at 17:00. Thursday has the highest average daily demand at roughly 18,098 trips, while Sunday has the lowest at about 12,059. The working-day afternoon peak is consistent with commuting and business activity. Median distance and fare rise during the low-demand early morning, when airport trips plausibly represent a larger demand share.

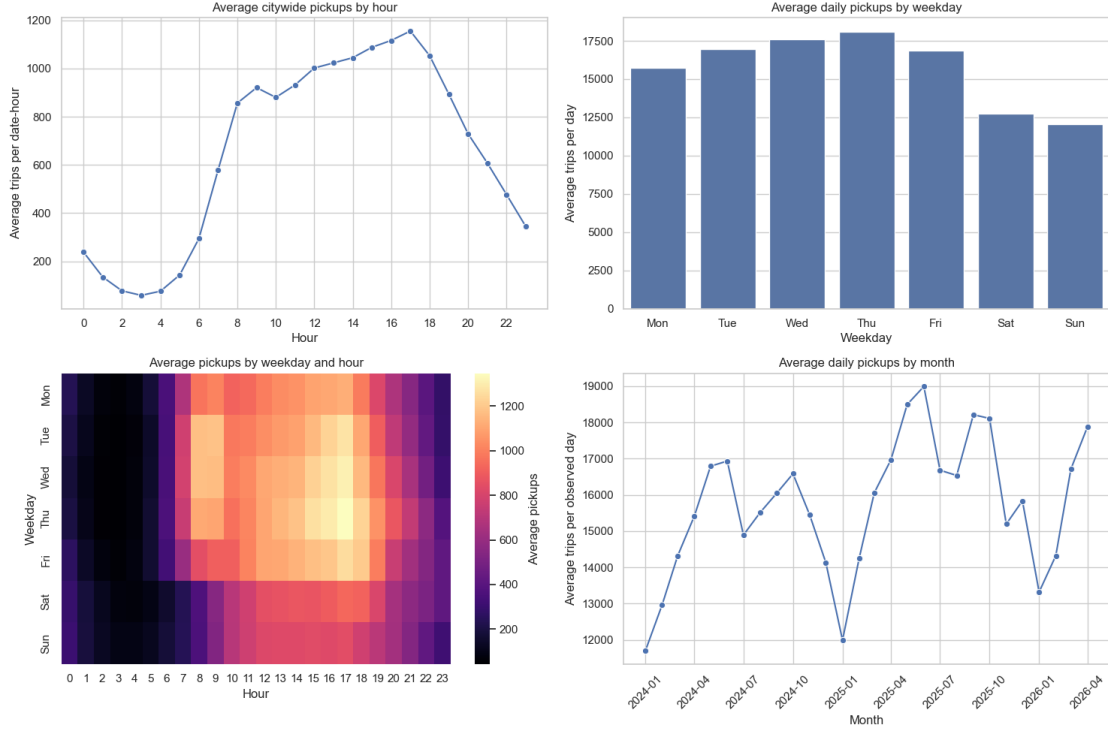


Figure 1: Temporal pickup-demand patterns.

Near North Side, O’Hare, Loop and Near West Side dominate pickup demand. The five leading Community Areas account for 74.66% of mapped pickups and the leading ten for 86.10%; the full pickup/drop-off comparison is shown in Figure 7 in the Appendix. O’Hare records about 5.36 pickups for every published drop-off. The central pattern is consistent with employment, tourism and nightlife, while O’Hare represents a separate airport market. The pickup/drop-off ratio remains approximate because drop-off locations are less complete. Resolution strongly affects target sparsity. Among Community Area-hour combinations, 91.58% have zero demand; the shares are 98.03% for Census Tracts and 98.52% for H3 resolution 8. Daily aggregation reduces these values to 76.76%, 93.42% and 95.07%, respectively; the full comparison is shown in Figure 8 in the Appendix. Finer units provide more local detail but many more zero observations. H3 resolution 7 is a middle ground, whereas Community Areas are easier to interpret but hide local variation. Observed inter-trip gaps have a median of 30 minutes and a mean of 68.45 minutes, but rounded timestamps and unknown shifts prevent interpreting them as exact idle time (Figure 10). Operationally, vehicle availability should concentrate on central Chicago and O’Hare, particularly during weekday afternoons. Predictive results should also separate zero-, low- and high-demand cases, since a low overall error can hide poor performance in the locations where capacity matters most.

4.2 GMM Hot-Spot Analysis

The Community Area map ranks fixed administrative units. To obtain a continuous demand surface, we fit Gaussian Mixture Models to projected pickup coordinates with scikit-learn (Scikit-learn Developers, [n.d.](#)). The fit uses the 6,049,435 trips with Census-based centroids; Community Area centroids are excluded because millions of observations repeated at only 77 coarse points would produce artificial peaks. No area label or trip attribute enters the model. Model selection uses a fixed 250,000-trip reservoir sample whose coordinate shares correlate at 0.999967 with the eligible population. We compare 116 candidates covering 2 to 30 components

and four covariance structures. The Bayesian Information Criterion is used for the initial ranking (Schwarz, 1978). The raw BIC winner collapses narrow components onto repeated centroids, so candidates must also converge, have at least 0.25% component weight and maintain a minimum standard deviation of 50 metres. These safeguards select tied covariance with 17 components; full diagnostics are shown in Figure 11 in the appendix. The selected GMM confirms the Community Area ranking but adds detail within the largest districts (Figure 2). O’Hare forms the largest component with 22.1% of sampled eligible demand. Near North Side, Loop, O’Hare and Near West Side together contain 90.2% of eligible trips in the administrative baseline, while the GMM places four component means in Near North Side, three in Loop and three in Near West Side. A Community Area total is therefore the simpler result for district-level planning; the GMM is useful for positioning vehicles within the large central demand areas and across administrative borders.

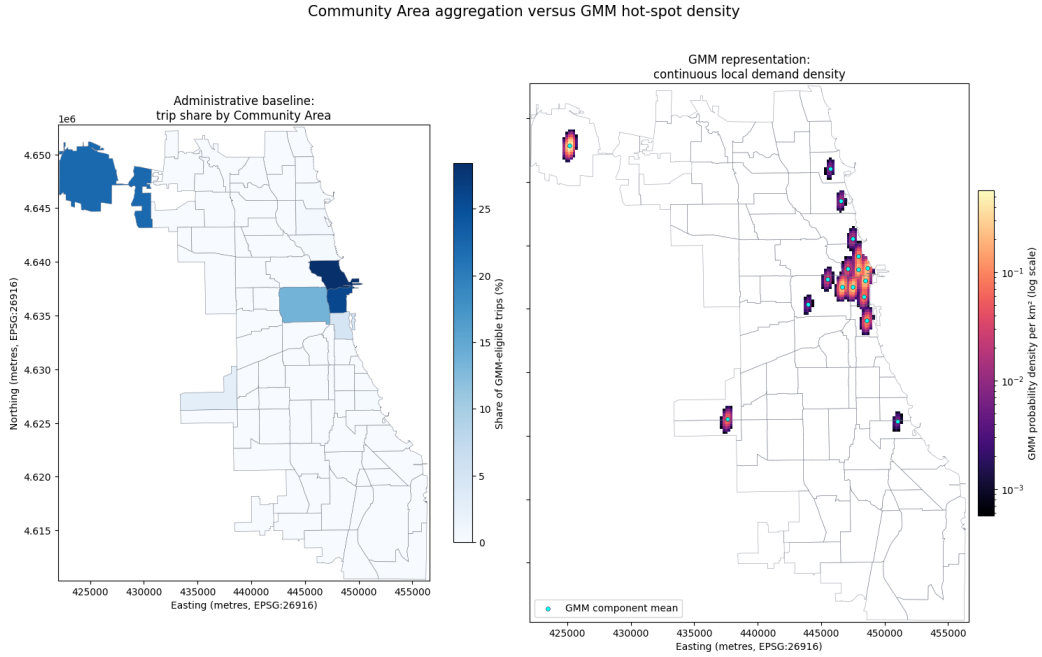


Figure 2: Community Area demand and selected GMM density.

The GMM still describes only 469 distinct published Census centroids, not street-level pickups. Its tied covariance stabilizes the fit but forces all components to share one ellipse shape. Components overlap and are density terms, not neighbourhoods or hard assignment zones. The result should therefore complement, rather than replace, the Community Area map.

4.3 NN

In the following sections we evaluate whether NN and SVM models improve demand prediction. The target, features and preprocessing are shown in Table 7. We compare the models with an always zero prediction and a spatial $\times$ time $\times$ weekday mean. The second baseline is stronger because it already represents the main recurring demand patterns. Our primary metric is MAE. It measures the average absolute difference between predicted and observed trips. The baseline results are shown in Table 12. The NN development consists of a baseline model, a two stage grid search and final retraining. The baseline model was trained for every spatial and temporal configuration. It uses two hidden layers with 64 and 32 nodes, ReLU activations and a spatial bias. Numerical inputs were standardized using training data only. The full architecture and training configuration are reported in Table 8 and Table 9. The best baseline checkpoint for each configuration was selected by validation MAE. Raw MAE values cannot be compared

directly across aggregation levels because the demand scale changes. We therefore also use an MAE Skill Score relative to the spatial $\times$ time $\times$ weekday baseline. A positive value means that the NN improves on this baseline. Only Census Tracts at 1 hour resolution achieve a positive score. The improvement is very small at 0.0043. The 24 hour models perform worst. This may partly result from their smaller training sets. Diagnostics for Census Tracts at 1 hour also show that the model has particular difficulty with high demand. The complete baseline results and diagnostics are shown in Table 11 and Figure 14. These findings led us to focus on positive demand during the grid search. We used Balanced MAE for model selection. It gives equal importance to zero and positive demand observations. Stage one compared standard training, positive demand weighting and undersampling. The tested configurations are reported in Table 15. All stage one models remained below the spatial $\times$ time $\times$ weekday baseline. However, weighting performed consistently better than undersampling. Stage two therefore combined the most promising weighting strategies with different architectures and learning rates. The search space and results are shown in Table 16 and Table 17. Model S2-7 performed best in the grid search. It uses hidden layers with 128 and 64 nodes, a learning rate of 0.001 and a weight of 5 for positive demand. It reached a validation Balanced MAE of 2.7877 and an MAE Skill Score of 0.0014. The best result occurred in epoch 29 of 30. This supported retraining with a larger epoch budget. Its training history is shown in Figure 18. The final model uses this configuration with up to 100 epochs and an early stopping patience of 10. The checkpoint selected at epoch 26 achieved a validation Balanced MAE of 2.9319. On the TEST set, the model achieved a Balanced MAE of 2.8775 and an MAE Skill Score of 0.0253. This represents only a small improvement over the spatial $\times$ time $\times$ weekday baseline. The difference between demand groups is very important for operations. The zero demand MAE is 0.0123, while the positive demand MAE is 5.7427. Peak demand MAE rises to 17.1399. Despite positive demand weighting, the model remains much more accurate when no trips occur. Precision is 0.8123 and recall is 0.7162. The model is therefore relatively reliable when it predicts demand, but it misses part of the observed demand. The full TEST results are shown in Table 13 and Figure 15. The selected configuration was also trained on the other spatial and temporal datasets. However, its settings were tuned only on Census Tracts at 1 hour resolution. The comparison in Figure 5 therefore shows how well the configuration transfers. It does not compare separately optimized models for every dataset. Improvements through lagged demand and a two stage prediction approach are discussed in the outlook.

4.4 Support Vector Machine

Support Vector Regression (SVR) provides the second demand-prediction approach. It predicts trip count for Community Areas (City of Chicago, 2025), Census Tracts (City of Chicago, 2013) and H3 hexagons (Uber Technologies, n.d.) at one-, four- and twenty-four-hour resolutions. The models use the previously described temporal, weather (Iowa Environmental Mesonet, n.d.), holiday and point-of-interest features. To provide one spatial representation across all three unit types, unit centroids are converted from longitude and latitude to spherical coordinates. The final scikit-learn pipeline (Buitinck et al., 2013; Pedregosa et al., 2011) scales inputs and the target, compares linear SVR with kernel approximations and uses successive-halving model selection. Kernel approximation, caching and restricted search spaces make the large data sets computationally tractable. GPU training with cuML (RAPIDS Development Team, 2023; Raschka et al., 2020) was also evaluated, but project runs on larger samples produced unstable results. The pipeline and this implementation-specific observation are documented in the Appendix.

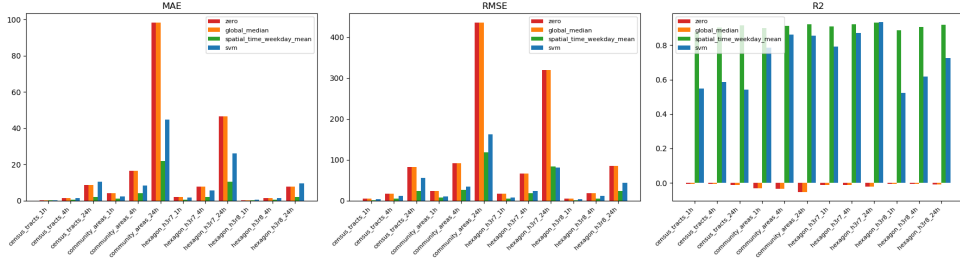


Figure 3: Test-metric comparison of SVR and the three baseline models.

4.4.1 Results

Across configurations, SVR generally outperforms the zero and global-median baselines but remains behind the spatial-time-weekday mean (Figure 3). The H3 resolution 7 four-hour model performs best relative to this strongest baseline, although the difference is small. The high share of zero-demand observations is a central limitation: aggregate errors can be dominated by easy zero cases, while sparse non-zero cases make model selection and fitting less stable.

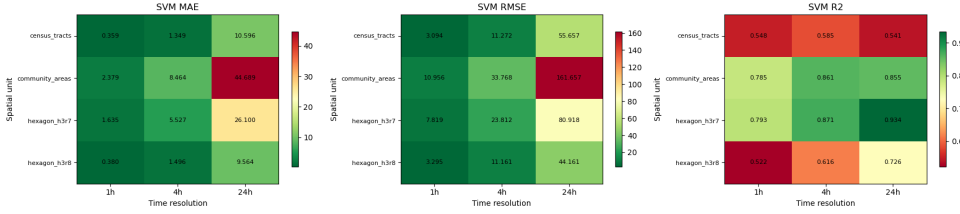


Figure 4: SVR metrics across spatial and temporal configurations.

The hexagonal representations provide the strongest overall results (Figure 4). H3 resolution 8 has lower absolute errors, whereas the coarser H3 resolution 7 performs better on R^2 . Census Tracts have the weakest R^2 , while Community Areas have the highest MAE and RMSE. Shorter time intervals generally perform better; the Community Area model is particularly weak at the twenty-four-hour resolution.

4.4.2 SVC

An exploratory Support Vector Classification (SVC) model maps demand to five data-informed but fixed classes: zero, low, medium, high and very high. Training uses class balancing to prevent zero demand from dominating. Performance was not strong enough to justify further development, and any operational application would require retuning the class boundaries; the exploratory result is shown in Figure 19 in the Appendix.

4.4.3 Further Improvement

Future work should first remove spatial units outside the operational area and evaluate the remaining zero-demand structure explicitly. Suitable alternatives include a two-stage classifier-regressor such as `ZeroInflatedRegressor` (Warmerdam et al., 2026) or a count model such as `ZeroInflatedPoisson` (Perktold et al., 2024). Model families should be screened on representative samples before full fitting so that computationally infeasible candidates are rejected early. Given the observed convergence, runtime and predictive performance, SVMs are not recommended as the primary model family for this demand data.

4.5 NN compared to SVM

We compare both models on the same TEST observations. The NN performs better on most error metrics and has higher precision. The SVM achieves higher recall across all configurations. It detects more demand intervals, but it also produces more false demand signals. The NN is more conservative, although the SVM performs better on several metrics for H3 resolution 7 at 24 hours. The number of metric wins is reported in Table 14. The demand bucket comparison provides a more detailed view. For Census Tracts at 1 hour, the SVM has slightly lower MAE for intervals with 1 or 2 to 3 trips. The NN performs better from 4 trips onwards. Its advantage grows at high demand. These results are shown in Figure 17. The complete comparison across datasets is presented in Figure 16. Both models usually improve on the always zero prediction. This is a weak benchmark because it does not support useful fleet operations. The stronger spatial $\times$ time $\times$ weekday baseline is only outperformed by the NN for Census Tracts at 1 hour. With the current data and features, the additional complexity of both model pipelines creates little operational value.

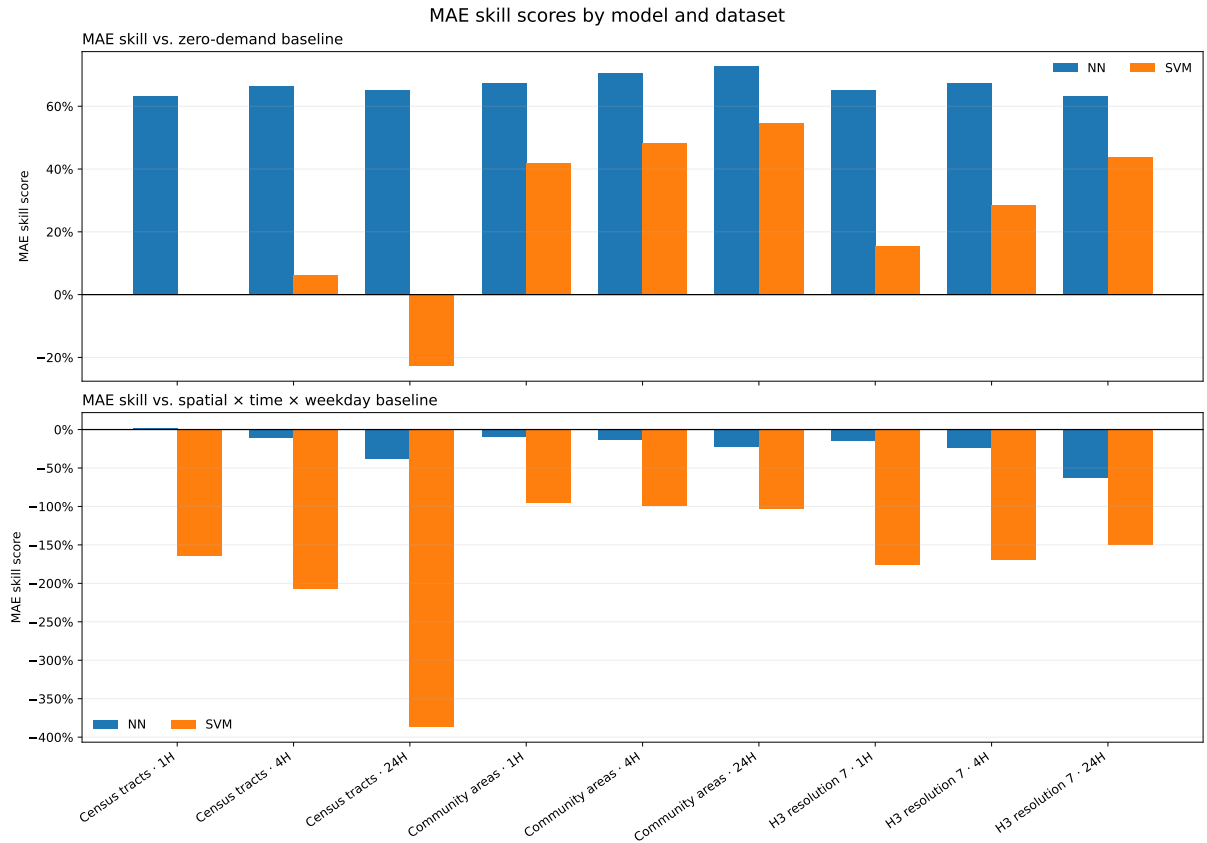


Figure 5: Test MAE skill scores of NN and SVM models against both baselines.

For the pilot, we therefore recommend the spatial $\times$ time $\times$ weekday baseline. It is simple, transparent and inexpensive to maintain. The current NN and SVM pipelines should not be used as the only basis for fleet dispatch. This result does not show that NNs are generally better than SVMs. It only applies to the pipelines and data evaluated in this report. A follow up project with additional information and a model designed for different demand levels may provide more value.

5 Smart Charging Reinforcement Learning

5.1 Motivation and Setup

The daily operation of electric vehicles requires them to recharge daily. Depending on the time of day, charging costs can vary significantly, especially between peak and off-peak hours. Therefore, charging cost is a significant operational expense for an electric fleet. A naive strategy of always charging at maximum power ensures the battery is full, but comes at an unnecessarily high cost since the cost grows exponentially with power. However, if we assume the vehicle is home for a fixed window, for example between 2pm and 4pm, we can be smart about when and how much to charge. With reinforcement learning we can simulate this smart charging behaviour, where an agent learns to balance cost savings against the risk of running out of energy. We model this as a Markov Decision Process (MDP), where the agent observes the current time and battery level, picks a charging power level, and receives feedback in the form of a cost signal.

Our MDP is defined as follows:

- State: the current 15-minute timestep (2:00pm, 2:15pm, ..., 4pm) and the current battery level.
- Action: one of 4 discrete charging power levels: 0 kW (off), 7 kW (low), 14 kW (medium), 22 kW (high).
- Transition: the battery increases by the chosen power $\times 0.25\text{h}$, capped at battery capacity.
- Reward: negative charging cost each step. At the final step (4pm), a random energy demand is drawn from a normal distribution ($\mu = 30$ kWh, $\sigma = 5$ kWh). If the battery cannot cover this demand, a large penalty is applied.

To ensure realistic simulation parameters, we grounded our assumptions in real-world data. The Tesla Model Y is one of the most commonly used models for EV taxis in North America (Kaiyi Global, 2026), and according to current EV specifications, a usable battery capacity of 60 kWh is representative of this vehicle class (EV Database, 2026). For the available charging levels, we considered Level 2 home chargers, which support power rates of up to 22 kW, sufficient for the Tesla Model Y (Pod Energy, 2026). Furthermore, we discretized the charging options into four levels: 0 kW (off), 7 kW (low), 14 kW (medium), and 22 kW (high). The starting battery at 2pm is sampled randomly between 5 and 20 kWh, reflecting a vehicle that arrives partially depleted after a morning shift. For the charging cost, we assumed home charging in Chicago, where residential electricity rates range from \$0.11 to \$0.16 per kWh (BeeCharge, 2026). We calibrated the cost coefficients to match these rates: at medium power (14 kW, delivering 3.5 kWh per 15-minute step), the coefficient ranges from 0.052 at the cheaper end to 0.076 at the more expensive end. The coefficient increases linearly across the eight timesteps, reflecting time-of-use pricing where electricity becomes more expensive during late afternoon hours.

We solve this using Q-Learning, a standard reinforcement learning method that learns optimal decisions through trial and error. Q-Learning is well suited here because the state space is small and discrete, allowing the agent to learn a complete policy without the overhead of a neural network. We additionally implemented a Deep Q-Network (DQN) to test whether a neural network-based approach could match or improve on the tabular solution. While Q-Learning works well for our small state space, DQN would scale better to more complex scenarios, for example if the charging window were longer or if real-time electricity prices were included as an additional state variable.

5.2 Results

We trained both agents for 20,000 episodes and evaluated the learned policies over 2,000 simulated episodes against a naive baseline that always charges at maximum power (22kW). The results are summarized below:

Table 1: Evaluation results over 20000 episodes.

Policy	Avg. Cost	Savings	Avg. Battery at 4pm	Out-of-Energy Rate
Baseline (always max)	10.14	—	55.9 kWh	0.0%
Q-Learning	6.38	37.1%	47.1 kWh	0.0%
DQN	8.24	18.8%	50.2 kWh	0.0%

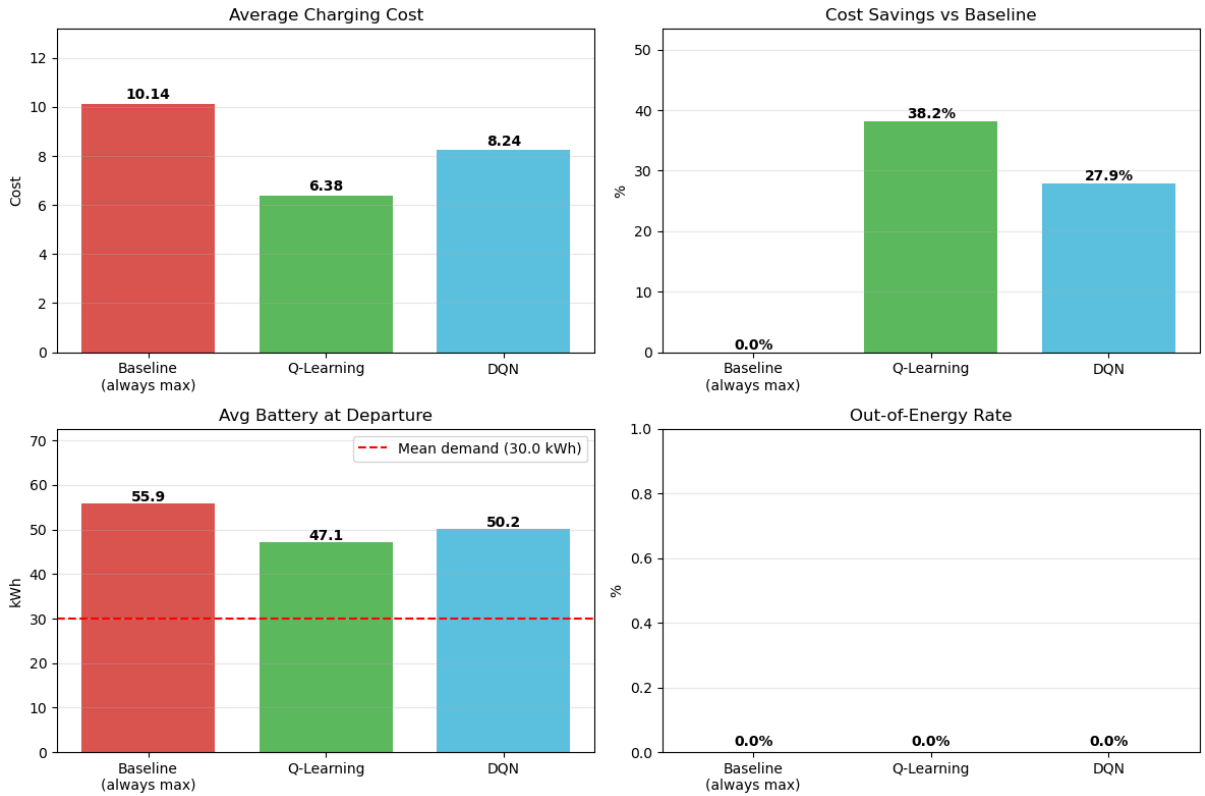


Figure 6: Comparison of charging policies: Q-Learning vs DQN vs Baseline.

The Q-Learning agent achieves a 37.1% cost reduction compared to the baseline while maintaining a 0% out-of-energy rate. It does this by charging more aggressively during the early, cheaper timesteps and reducing or skipping charging later when cost coefficients are higher. The average battery departure (47.1 kWh) comfortably exceeds the mean demand of 30 kWh, providing a safety margin against demand uncertainty without overcharging. A representative charging schedule and battery trajectory are shown in Figure 20 in the Appendix. The DQN agent also outperforms the baseline with an 18.8% cost reduction, though it charges more conservatively than Q-Learning, resulting in a higher average battery at departure (50.2kWh). This is expected since the tabular Q-Learning approach can represent the optimal policy exactly for our small, discrete state space. Meanwhile the DQN introduces approximation, however, its advantage lies

in its scalability, as it could handle a more complex scenario with continuous state variables or a longer charging window without modification. Its representative charging schedule is shown in Figure 21 in the Appendix. All three policies achieve a 0% out of energy rate, confirming that the penalty mechanism successfully teaches the agents to prioritize energy sufficiency. This is substantial, since this could leave the driver stranded and lose a full day of revenue. The results demonstrate that even in a simplified two-hour charging window, an intelligent charging agent can reduce costs by over a third compared to a naive strategy. Scaled to an entire fleet, these savings become significant. For example, if a fleet of 100 vehicles each charges once per day, a 37% reduction in per-vehicle charging cost translates directly into lower daily operating expenses. The smart charging system is fully automated and requires no manual intervention from drivers or dispatchers. Once trained, the agent makes real-time charging decisions based on the current battery level and time of day, adapting naturally to varying starting conditions. Moreover, the approach is flexible. If the electricity pricing structures change or the charging window shifts or the parameters are changed, the agent can simply be retrained on the updated parameters. This makes it a future-proof component of the fleet’s operational toolkit, complementing the demand prediction models developed in earlier sections of this analysis.

6 Conclusion

First of all, an average of 15,731 daily taxi trips indicates substantial demand and makes Chicago a promising market. The data analysis has identified spatial and temporal demand patterns, for example, we found that the five busiest community areas account for 74.66% of pickups, demand peaks on thursdays around 5 pm, while the demand is lowest on sundays. Further trips from and to the airport are very frequent. For our customer this means that they should initially focus on the busiest community areas in the central Chicago and availability should be adapted to weekday and hour. With the current data and restrictions on feature usage the analysis showed that NN and SVM models create little operational value. Only the NN on census_tract_1h outperforms the spatial x time x weekday baseline, while prediction errors are largest during high-demand periods. Without additional data sources, lag-features or an ensemble model using a demand classifier and separate high and low demand models, the simple and transparent baseline is preferable. Regarding smart charging, we found that Q-learning reduces charging cost by 37.1% while maintaining a 0% out-of-energy rate. Its tabular approach is more suitable than the DQN for the small discrete state spaces. The simulation assumes private charging and excludes installation costs, public-charger queues and lost revenue. Its 2 pm – 4 pm charging window may also conflict with the 5 pm demand peak. Therefore, we recommend our customer to initially combine public charging as backup with limited private depot charging at locations with predictable dwell time. Further investment requires a total-cost comparison covering infrastructure, electricity, waiting time, empty mileage and lost trips. Lastly, we want to provide a methodological assessment and outlook. Complete spatial-time panels distinguish observed zero-trip periods from missing observations. Separate error measures for zero and non-zero demand together with a meaningful baseline, prevent the many zero-demand periods from overstating model performance. Nevertheless, privacy-protected centroids prevent street-level conclusions, and competition and fleet economics are not observed. In a follow-up project lagged demand, chronological testing and a two-stage classifier-regression approach should address zero, low and high demand. Fare and drop-off data could support revenue and repositioning analyses. Further additional data sources like events, airport schedules, actual ride-hailing requests, charger availability and electricity tariffs would improve operational decisions. Our final recommendation is, that Chicago shows high potential as a promising follow-up project and electric-fleet pilot. However, it is not ready for an unconditional city-wide rollout yet. The expansion should depend on vehicle utilization, charging economics and contribution margins.

7 Appendix

7.1 Supplementary Data and Preparation Material

Table 2: Data sources used in the analysis.

Source	Provider	Coverage	Role
Taxi trips	Chicago Data Portal	2024-01-01 to 2026-04-30	Trips and pickup demand
Spatial boundaries	Chicago Data Portal	Chicago	Community Areas and Census Tracts
Weather	Iowa Environmental Mesonet	MDW, ORD and IGQ	Hourly weather context
Points of interest	OpenStreetMap via OSMnx	Chicago	Spatial context

Table 3: Main plausibility rules applied to taxi trips.

Variable	Retained range	Purpose
Distance	$0 < miles \leq 250$	Remove zero and implausibly long trips
Duration	$59 < seconds \leq 10,800$	Retain trips of about 1 minute to 3 hours
Fare	$0 < fare \leq 500$	Require a positive plausible fare
Tips	$0 \leq tips \leq 200$	Remove negative or extreme values
Tolls	$0 \leq tolls \leq 100$	Remove negative or extreme values
Extras	$0 \leq extras \leq 100$	Remove negative or extreme values
Trip total	$0 < total \leq 1,000$	Require a positive plausible total

Table 4: Spatial representations used in the analysis.

Spatial representation	Units	Geometry	Interpretability
Community Areas	77	Administrative	High
Census Tracts	878	Administrative	Medium
H3 r7	163	Regular grid	Medium
H3 r8	976	Regular grid	Low

Table 5: Distributional summary of the complete cleaned trip population.

Metric	Mean	Median	p95	p99
Distance (miles)	7.15	3.91	18.41	26.80
Duration (minutes)	21.06	16.05	53.55	75.00

Metric	Mean	Median	p95	p99
Fare (USD)	22.07	15.25	49.50	69.75
Trip total (USD)	27.11	17.81	65.62	96.00

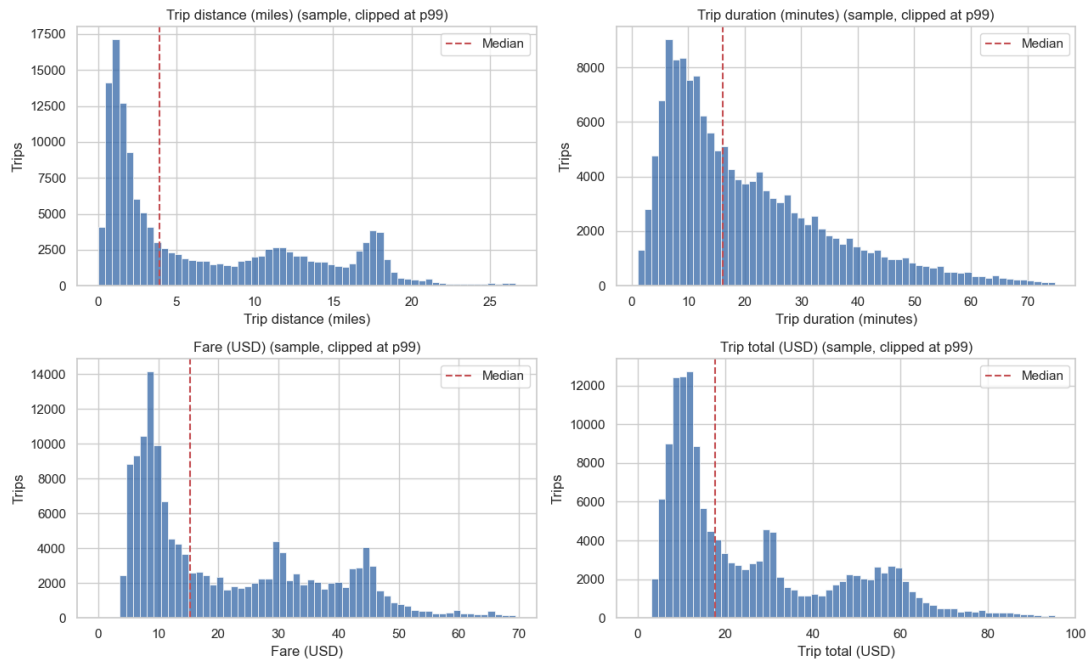


Figure 9: Distributions of trip distance, duration, fare and total payment in a fixed 150,000-trip reservoir sample of the complete cleaned population. Values above the sample p99 are omitted from each panel for readability.

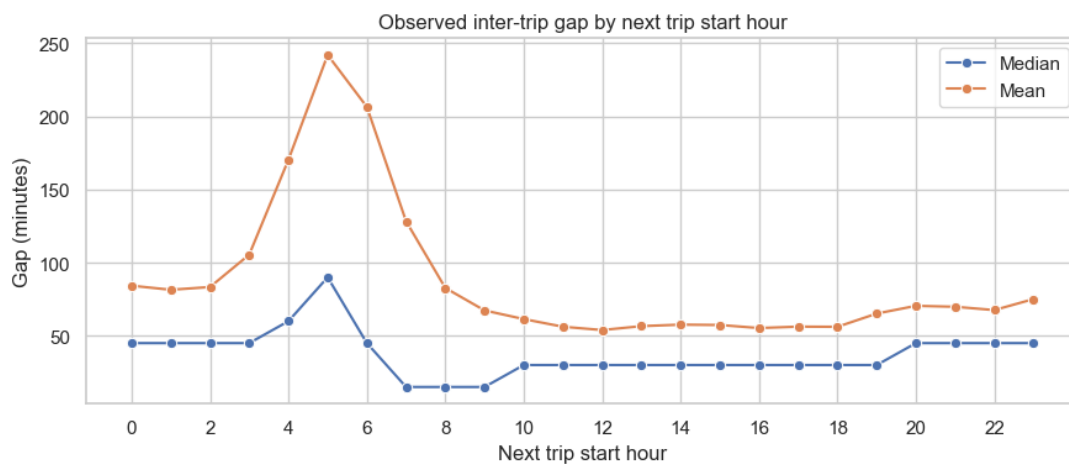


Figure 10: Hourly mean and median of retained gaps between consecutive observed trips of the same anonymized taxi.

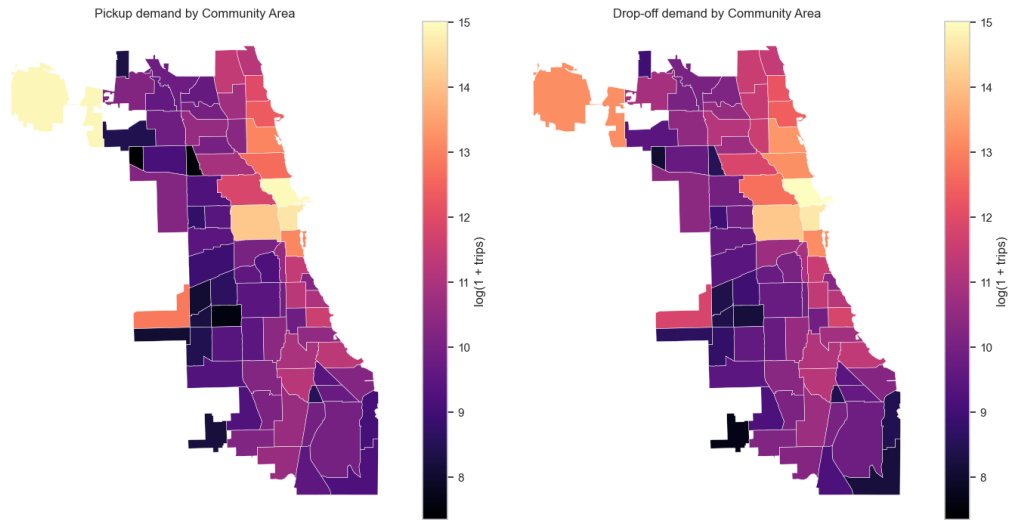


Figure 7: Pickup and drop-off demand by Community Area.

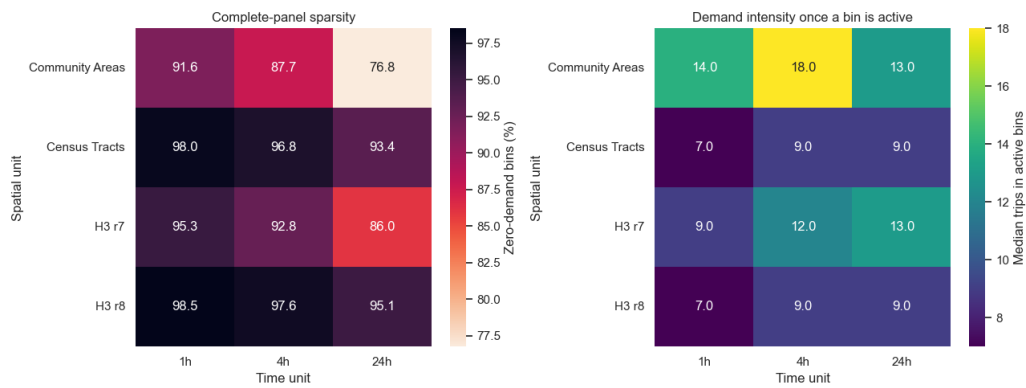


Figure 8: Demand sparsity by spatial and temporal resolution.

7.2 Supplementary GMM Diagnostics

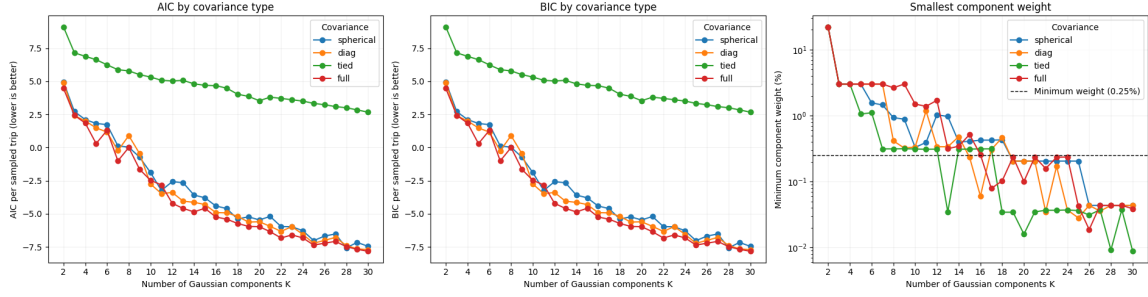


Figure 11: GMM candidate comparison across covariance structures and K=2–30. The selected tied-covariance model with K=17 is the best BIC candidate that satisfies the component-spread and minimum-weight safeguards.

Table 6: Largest components of the selected tied-covariance GMM.

Component	Community Area at mean	Weight
1	O'Hare	22.1%
2	Near West Side	12.1%
3	Loop	11.9%
4	Loop	11.2%
5	Near North Side	11.1%
6	Near North Side	9.4%
7	Near South Side	5.0%
8	Near North Side	4.9%

All components share a major standard deviation of 0.356 km, a minor standard deviation of 0.162 km and a 95% ellipse area of 1.085 km².

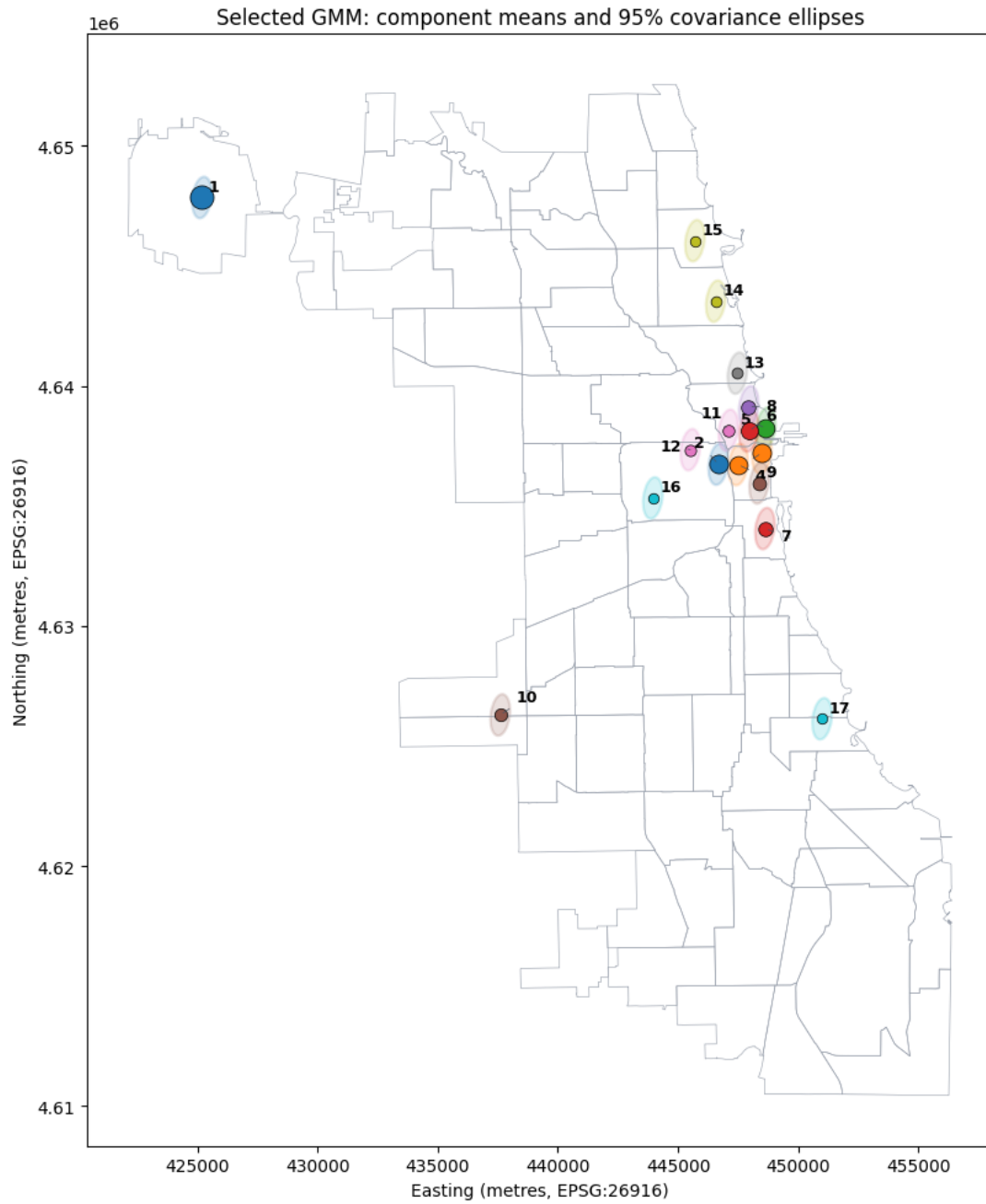


Figure 12: Component means and 95% probability ellipses of the selected tied-covariance GMM. Ellipses are contours rather than hard assignment boundaries.

Modelled pickup-demand density by six-hour window

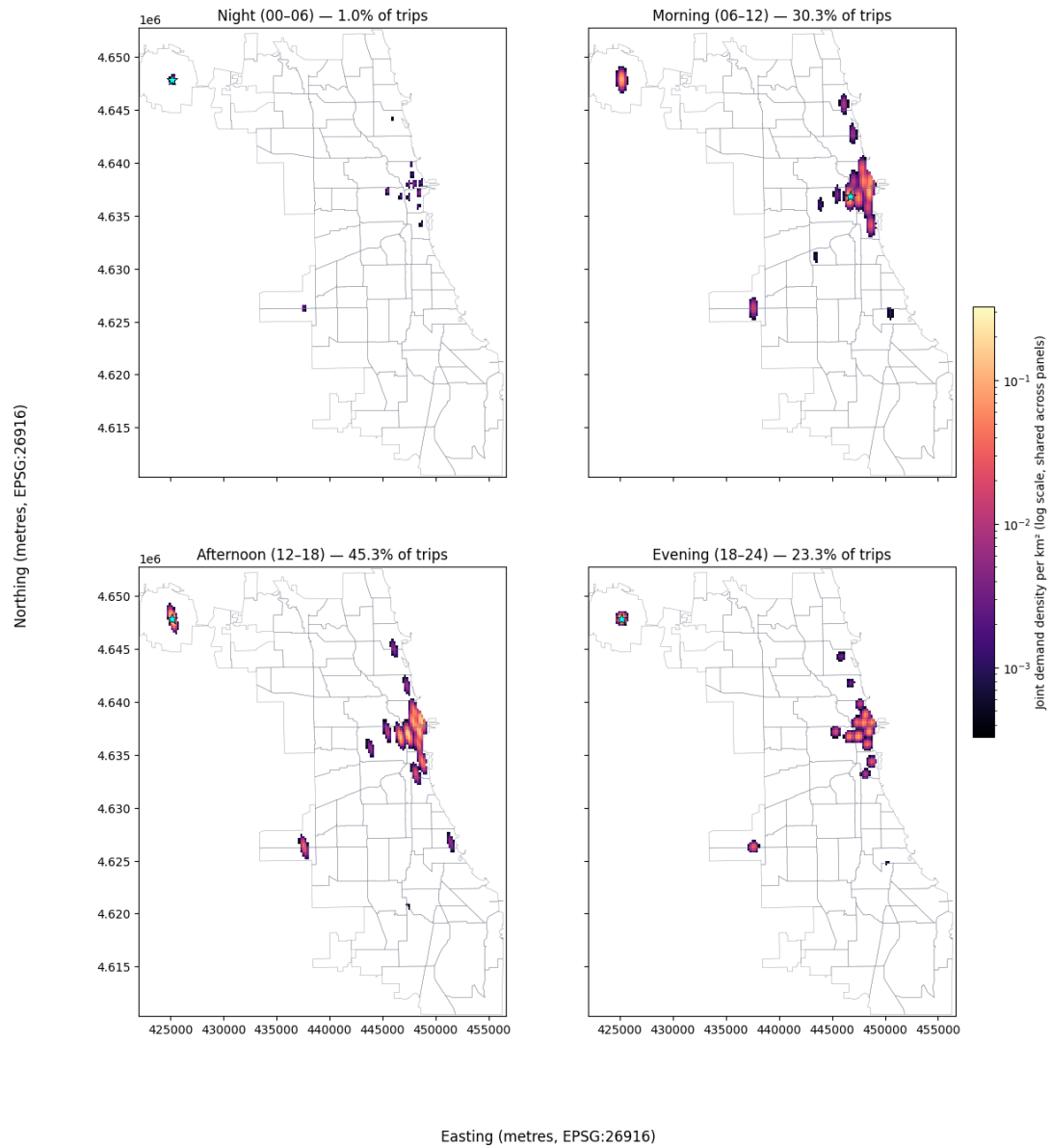


Figure 13: GMM density surfaces for four six-hour windows using the selected spatial model structure.

7.3 Neural-Network Details

Table 7: Target, features and preprocessing for NN training.

Role / feature group	Variable(s)	Preprocessing
Target	trip_count	Converted to float32; not scaled
Spatial unit	census_tract, h3_cell or community_area	TRAIN-only categorical index; unseen units map to index 0
Cyclical time	month_sin, month_cos, weekday_sin, weekday_cos, hour_sin, hour_cos	Cyclically encoded, then standardized
Calendar	is_holiday	Binary indicator, then standardized
Points of interest	food_drink, landmark, shop, train_station	Counts per spatial unit, then standardized
Weather measurements	tmpc, relh, sknt, p01m, vsby, wind_dir_sin, wind_dir_cos	Quality-controlled and imputed before TRAIN-only standardization
Weather conditions	weather_rain, weather_snow, weather_fog_mist, weather_thunder, weather_freezing, precipitation_trace	Binary indicators, then standardized
Cloud cover	skyc1_CLR, skyc1_FEW, skyc1_SCT, skyc1_BKN, skyc1_OVC, skyc1_VV	One-hot indicators, then standardized
Weather-station context	weather_station_distance_km, weather_station_MDW, weather_station_ORD, weather_station_IGQ	Distance and one-hot station indicators, then standardized
H3 metadata	h3_resolution	Included only for H3 datasets, then standardized

Table 8: NN baseline architecture.

Component	Input dimension	Output dimension	Activation
Numeric input	n_{features}	n_{features}	—
Hidden layer 1	n_{features}	64	ReLU
Hidden layer 2	64	32	ReLU
Numeric output	32	1	Linear
Spatial bias	Spatial-unit ID	1	—
Combination	Numeric score + spatial bias		1 Addition
Final output	1	1	Softplus

The spatial bias is implemented as a one-dimensional embedding. It represents an additive spatial intercept rather than a multidimensional representation-learning embedding. Index zero is reserved for spatial units not observed in the training split and has no learned effect.

7.4 Neural-network baseline training configuration

Table 9: NN baseline training configuration.

Setting	Configuration
Datasets	H3 resolution 7, census tracts, and filtered community areas
Temporal units	1h, 4h, and 24h
Number of models	9
Training objective	Mean squared error (MSE)
Optimizer	Adam
Learning rate	0.001
Batch size	1,024
Maximum epochs	100
Early-stopping patience	10 epochs
Minimum improvement	0
Checkpoint-selection metric	Validation MAE
Output constraint	Softplus activation
Numerical preprocessing	StandardScaler fitted exclusively on TRAIN
Spatial preprocessing	Categorical mapping fitted exclusively on TRAIN
Reproducibility	Dataset-specific seed, starting at 42
Model development split	TRAIN for fitting and VAL for model selection
Final evaluation split	TEST remains unused during model development

7.5 Neural-network baseline validation results

Table 10: NN baseline validation results.

Model	NN MAE	Naive baseline MAE	MAE skill score
Census Tracts, 1h	0.1396	0.1402	0.0043
Community Areas, 1h	1.3758	1.2853	-0.0705
Community Areas, 4h	5.4196	4.6133	-0.1748
Census Tracts, 4h	0.5584	0.4659	-0.1985
H3 r7, 1h	0.7556	0.6220	-0.2148
H3 r7, 4h	2.9352	2.2093	-0.3286
H3 r7, 24h	15.0754	11.1707	-0.3495
Census Tracts, 24h	3.3413	2.2846	-0.4625
Community Areas, 24h	100.9842	23.6039	-3.2783

Table 11: NN baseline validation results.

Model	NN MAE	$S \times T \times W$ MAE	Skill: zero	Skill: $S \times T \times W$
1 census_tracts_1h	0.1396	0.1402	0.6187	0.0043
2 community_areas_1h	1.3758	1.2853	0.6704	-0.0705

Table 11: NN baseline validation results.

	Model	NN MAE	S×T×W MAE	Skill: zero	Skill: S×T×W
8	community_areas_4h	5.4196	4.6133	0.6754	-0.1748
7	census_tracts_4h	0.5584	0.4659	0.6186	-0.1985
0	hexagon_h3r7_1h	0.7556	0.6220	0.6168	-0.2148
6	hexagon_h3r7_4h	2.9352	2.2093	0.6278	-0.3286
3	hexagon_h3r7_24h	15.0754	11.1707	0.6814	-0.3495
4	census_tracts_24h	3.3413	2.2846	0.6197	-0.4625
5	community_areas_24h	100.9842	23.6039	-0.0081	-3.2783

7.6 Trivial baseline test performance

Table 12: Test MAE of both baselines by dataset.

Dataset	Zero MAE	Spatial-time-weekday MAE
hexagon_h3r7_1h	1.9365	0.5924
census_tracts_1h	0.3595	0.1356
community_areas_1h	4.0993	1.2164
hexagon_h3r7_4h	7.7459	2.0454
census_tracts_4h	1.438	0.4397
community_areas_4h	16.3973	4.2469
hexagon_h3r7_24h	46.4757	10.442
census_tracts_24h	8.6282	2.1754
community_areas_24h	98.3836	21.9823

7.7 Neural-network baseline diagnostics

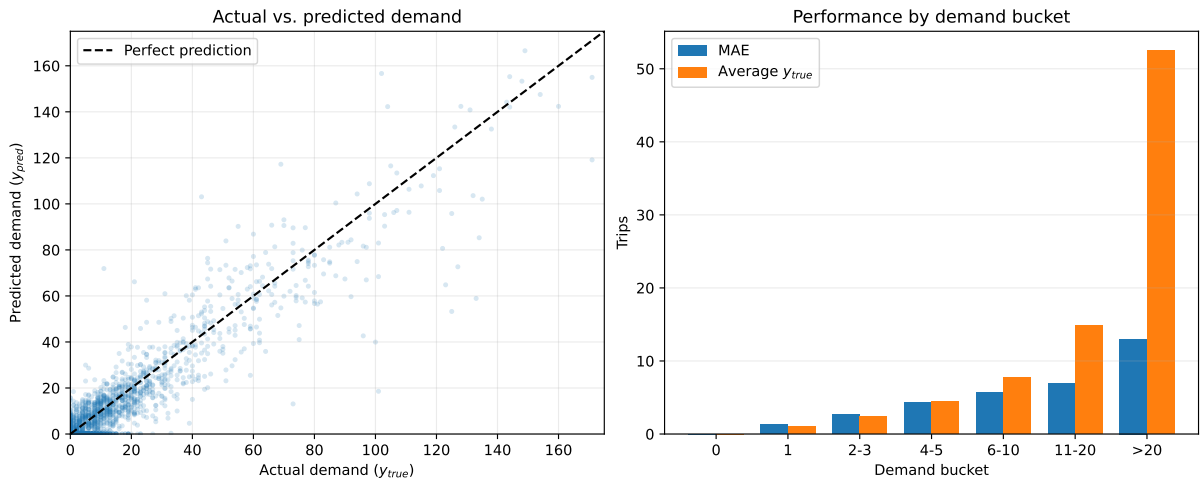


Figure 14: Validation diagnostics for the Census Tracts 1h NN baseline.

7.8 Final census-tract 1h test scores

Table 13: Final Census Tracts 1h NN test performance.

Metric	Value
MAE	0.1322
RMSE	1.4994
R ²	0.8939
Zero-demand MAE	0.0123
Non-zero-demand MAE	5.7427
Peak-demand MAE	17.1399
Balanced MAE	2.8775
MAE Skill Score vs. zero	0.6322
MAE Skill Score vs. spatial $\times$ time $\times$ weekday	0.0253
Demand precision	0.8123
Demand recall	0.7162

7.9 Final model test MAE by spatial unit

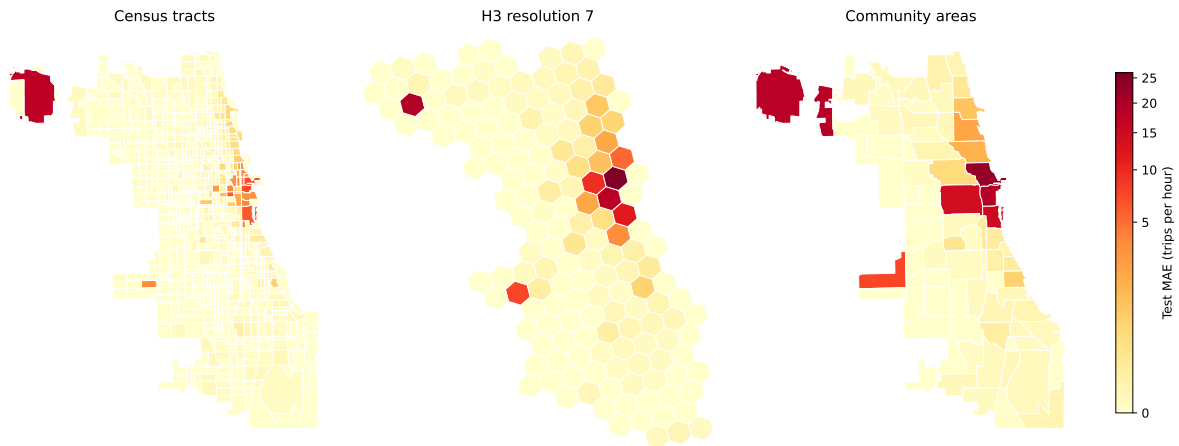


Figure 15: Test MAE of final 1h NN models by spatial unit.

7.10 Final NN and SVM pipeline comparison

Table 14: Test metric wins by model across all datasets.

Metric	Preferred value	NN wins	SVM wins
R ²	Higher	8	1
MAE	Lower	9	0
RMSE	Lower	8	1
Demand MAE	Lower	8	1
Zero-demand MAE	Lower	9	0
Balanced MAE	Lower	9	0
Peak-demand MAE	Lower	8	1
Demand precision	Higher	9	0
Demand recall	Higher	0	9
MAE skill vs. zero	Higher	9	0
MAE skill vs. spatial $\times$ time $\times$ weekday	Higher	9	0

Table 14: Test metric wins by model across all datasets.

Metric	Preferred value	NN wins	SVM wins
--------	-----------------	---------	----------

7.11 Neural-network grid search

Table 15: NN grid search models in stage 1.

Model	Strategy	Strategy parameter
S1-1	standard	–
S1-2	weighted_loss	positive weight = 2
S1-3	weighted_loss	positive weight = 5
S1-4	weighted_loss	positive weight = 10
S1-5	zero_undersampling	zero:positive = 1
S1-6	zero_undersampling	zero:positive = 2
S1-7	zero_undersampling	zero:positive = 5

Table 16: NN grid search models in stage 2.

Model	Positive weight	Hidden layers	Learning rate
S2-1	2.0	64-32	0.001
S2-2	2.0	64-32	0.0003
S2-3	2.0	128-64	0.001
S2-4	2.0	128-64	0.0003
S2-5	5.0	64-32	0.001
S2-6	5.0	64-32	0.0003
S2-7	5.0	128-64	0.001
S2-8	5.0	128-64	0.0003

Table 17: Validation results for all NN grid search models.

Stage	Model	Best epoch	Balanced MAE	MAE skill score
1	S1-1	20	3.4055	-0.1528
1	S1-2	19	3.1548	-0.0242
1	S1-3	20	3.1001	-0.0367
1	S1-4	15	3.018	-0.0797
1	S1-5	19	3.66	-0.9906
1	S1-6	19	3.6847	-0.4227
1	S1-7	20	3.5237	-0.2444
2	S2-1	24	3.0929	-0.0001
2	S2-2	29	3.3638	-0.0729
2	S2-3	16	3.0774	-0.0014
2	S2-4	29	3.4619	-0.0997
2	S2-5	29	3.0245	-0.1996
2	S2-6	29	3.3499	-0.0885
2	S2-7	29	2.7877	0.0014

Table 17: Validation results for all NN grid search models.

Stage	Model	Best epoch	Balanced MAE	MAE skill score
2	S2-8	24	3.422	-0.104

7.12 Grid-search winner training history

7.13 Appendix SVM

7.13.1 Bug found in cuml

Cuml(RAPIDS Development Team, 2023) appears to encounter challenges when confronted with substantial data sets, resulting in negative R2 values and a progressive decline in performance over time. This seems to happen at with datasets greater than 100_000. Other similar bugs seem to have been found before.

7.13.2 Steps in developing SVM

These are the changes we made during the developement of the svm in steps and why we decided on them: - We started with a basic svr on an rbf kernel which is the basic kernel for scikits(Buitinck et al., 2013; Pedregosa et al., 2011) implementation of svms. - Then we added grid search to find good hyperparameter. - SVM work better on scaled data, therefore we added a StandardScaler(Buitinck et al., 2013; Pedregosa et al., 2011). - Since we were running into runtime issues we changed changed grid search to only include the hyperparameter relevant for each kernel. - To further counteract these issues we added Nystroem(Buitinck et al., 2013; Pedregosa et al., 2011). - Due to the same reason, we changed GridSearchCV to RandomSearchCV(Buitinck et al., 2013; Pedregosa et al., 2011). - Since we were using a pipeline now we also added the StandardScaler to the Pipeline(Buitinck et al., 2013; Pedregosa et al., 2011). - Since we were still runnig into issues withe the runtime and to get a better performing SVR we added TransformedTargetRegressor(Buitinck et al., 2013; Pedregosa et al., 2011). - For the same reason as before we added caching(memory) to the pipeline. - The SVM with the kernel linear was changed to LinearSVM(Buitinck et al., 2013; Pedregosa et al., 2011). - Use cuml(RAPIDS Development Team, 2023, @raschka2020machine) to run model on the GPU instead of the CPU. - Find a good subset of hyperparameter through testing. - Changed from RandomSearchCV to HalvingGridSearchCV. Used RandomSearchCV before since HalvingGridSearchCV is still experimental. For our data HalvingGridSearchCV seems to work better(Buitinck et al., 2013; Pedregosa et al., 2011). - The scikit-learn LinearSVR, LinearSVC, SVR, and SVC were changed to the equivalent cuml version.(Buitinck et al., 2013; Pedregosa et al., 2011; RAPIDS Development Team, 2023; Raschka et al., 2020) - Changed it so y_pred can only be non-negative since trip count can not be negative.

7.13.3 Pipeline SVR/SVC

- TransformedTargetRegressor (not part of SVC)
- StandardScaler
- Nystroem
- SVR
- TransformedTargetRegressors StandardScaler

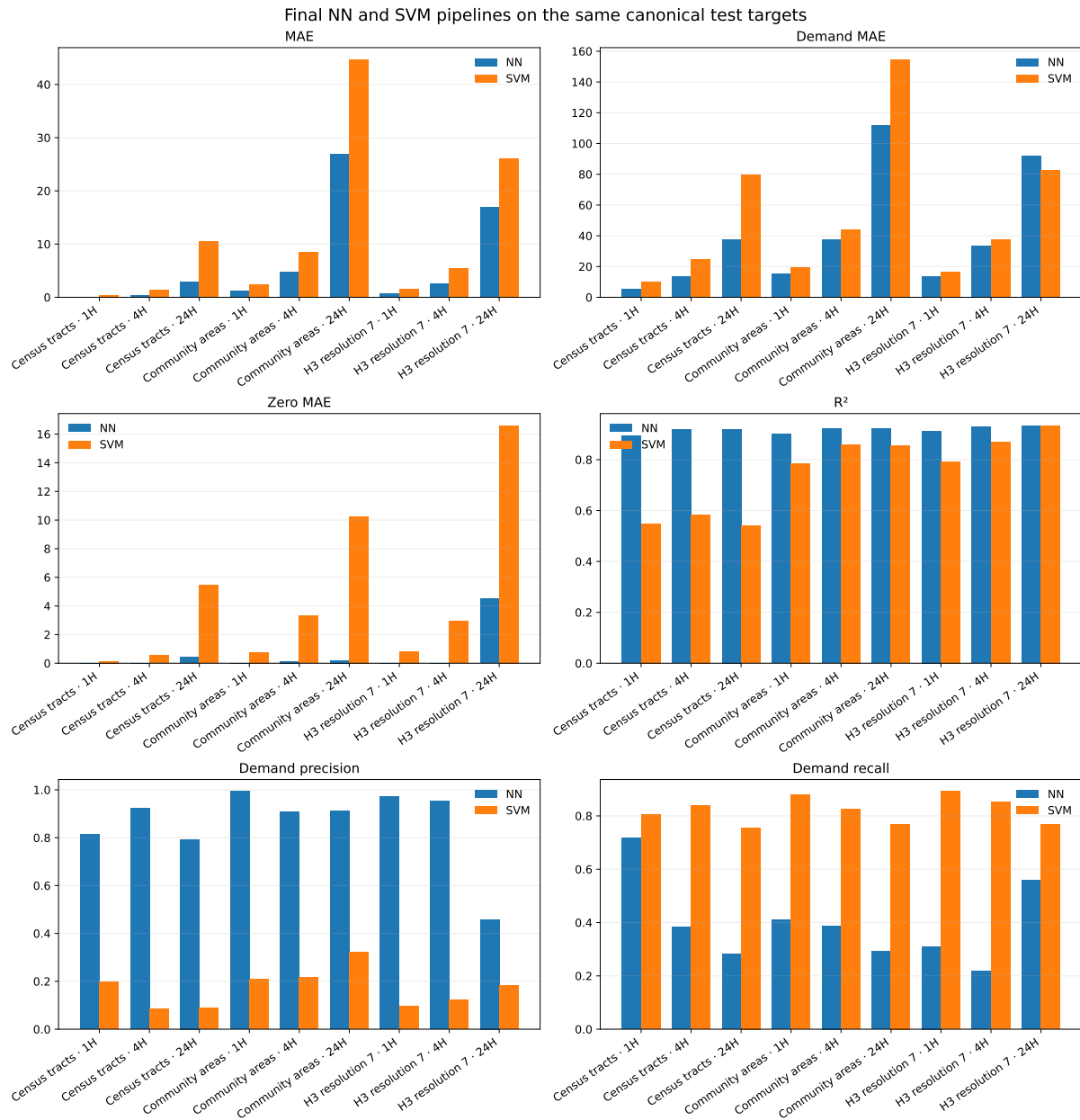


Figure 16: TEST comparison of the final NN and SVM pipelines.

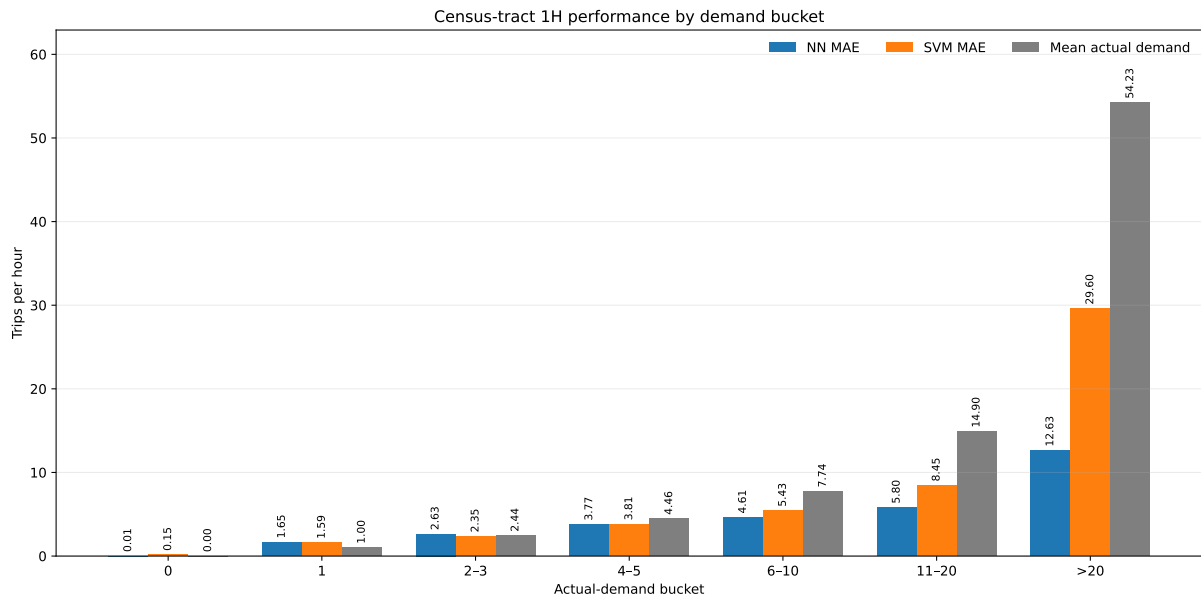


Figure 17: NN and SVM test MAE by demand bucket for Census Tracts 1h.

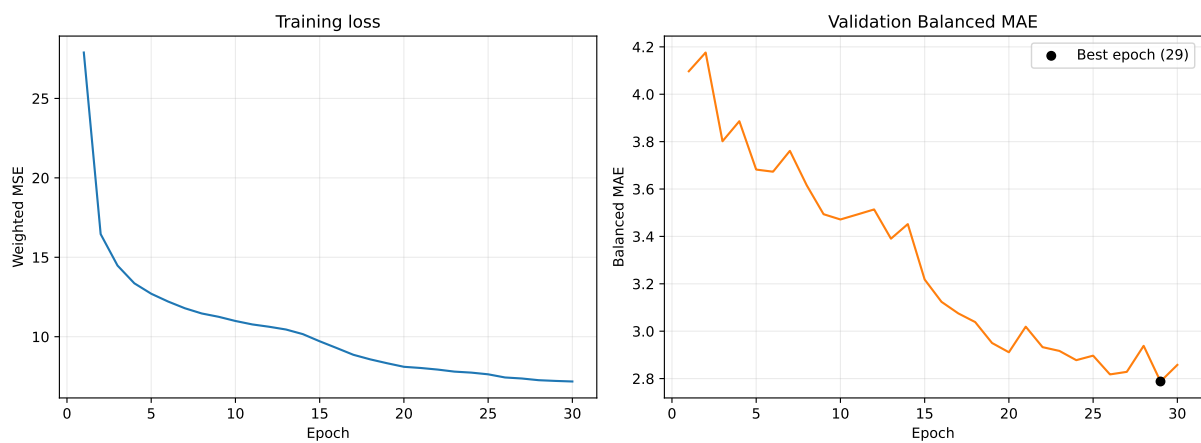


Figure 18: Training history of the selected NN grid search model.

7.13.4 Current Result SVC

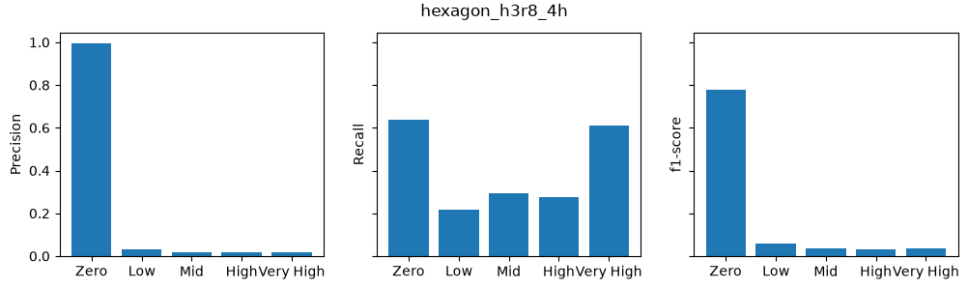


Figure 19: Statistics of SVC

The current result of the SVC does not look good. Due to the focus on the SVR, we were unable to further develop this model (Buitinck et al., 2013; Pedregosa et al., 2011).

7.14 Appendix: Reinforcement Learning

7.14.1 Cost Coefficient Calibration

The charging cost at each timestep follows an exponential form: $c_t = \alpha_t \cdot e^a$, where a is the discrete action index (0 = off, 1 = low, 2 = medium, 3 = high) and α_t is the time-varying cost coefficient.

We calibrated α_t to match real Chicago home charging rates of \$0.11–\$0.16 per kWh. At medium power (action = 2, corresponding to 14 kW and delivering 3.5 kWh per 15-minute step):

- Cheap end (\$0.11/kWh): $\alpha = 0.11 \times 3.5 / e^2 \approx 0.052$
- Expensive end (\$0.16/kWh): $\alpha = 0.16 \times 3.5 / e^2 \approx 0.076$

The coefficient increases linearly across the eight timesteps, yielding:

Table 18: Time-varying cost coefficients per timestep.

Timestep	2:00	2:15	2:30	2:45	3:00	3:15	3:30	3:45
α_t	0.052	0.055	0.058	0.061	0.064	0.067	0.072	0.076

7.14.2 Hyperparameters

Table 19: Hyperparameter settings for Q-Learning and DQN.

Parameter	Q-Learning	DQN
Episodes	20,000	20,000
Discount factor (γ)	0.99	0.99
Learning rate	0.1	0.001
Epsilon (start)	1.0	1.0
Epsilon (min)	0.01	0.01
Epsilon decay	0.9995	0.9995
Hidden layer size	—	64

Parameter	Q-Learning	DQN
Replay buffer size	—	20,000
Batch size	—	64
Target network update	—	every 100 episodes
Optimizer	—	Adam

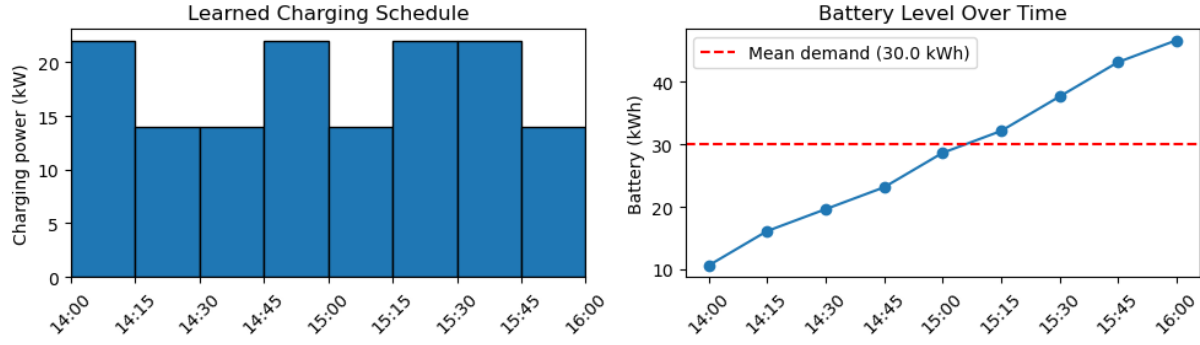


Figure 20: Learned Q-Learning charging schedule and battery trajectory.

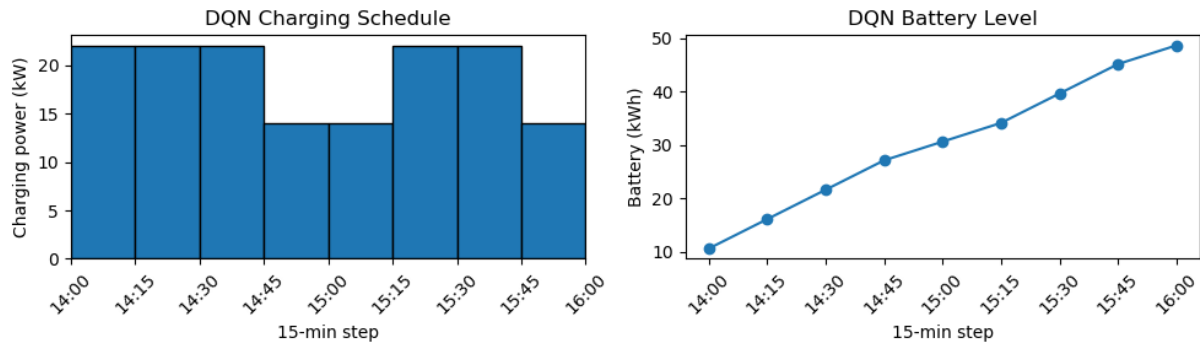


Figure 21: DQN charging schedule and battery trajectory.

7.14.3 Training Convergence

7.14.4 Learned Policy Heatmap

7.14.5 Cost Distribution

References

- BeeCharge. (2026). *Mobile ev charging cost in chicago, il*. <https://www.beechargedev.com/mobile-ev-charging-cost-in-chicago-il/>
- Buitinck, L., Louppe, G., Blondel, M., Pedregosa, F., Mueller, A., Grisel, O., Niculae, V., Prettenhofer, P., Gramfort, A., Grobler, J., Layton, R., VanderPlas, J., Joly, A., Holt, B., & Varoquaux, G. (2013). API design for machine learning software: Experiences from the scikit-learn project. *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, 108–122.
- City of Chicago. (2013). *Census tracts*. Retrieved July 26, 2026, from https://data.cityofchicago.org/Facilities-Geographic-Boundaries/Census_Tracts/4hp8-2i8z/about_data

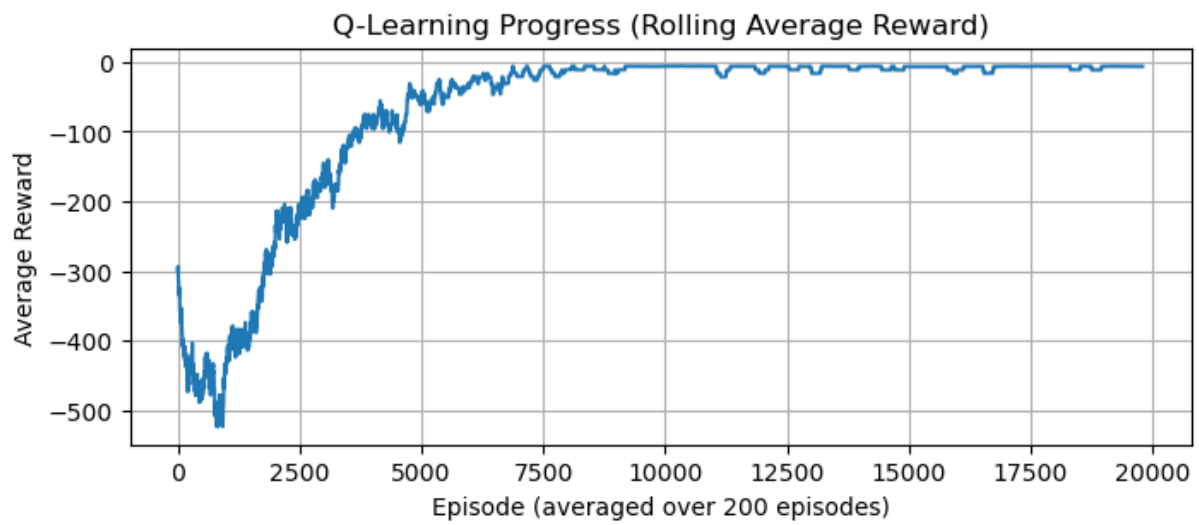


Figure 22: Q-Learning training progress (rolling average reward).

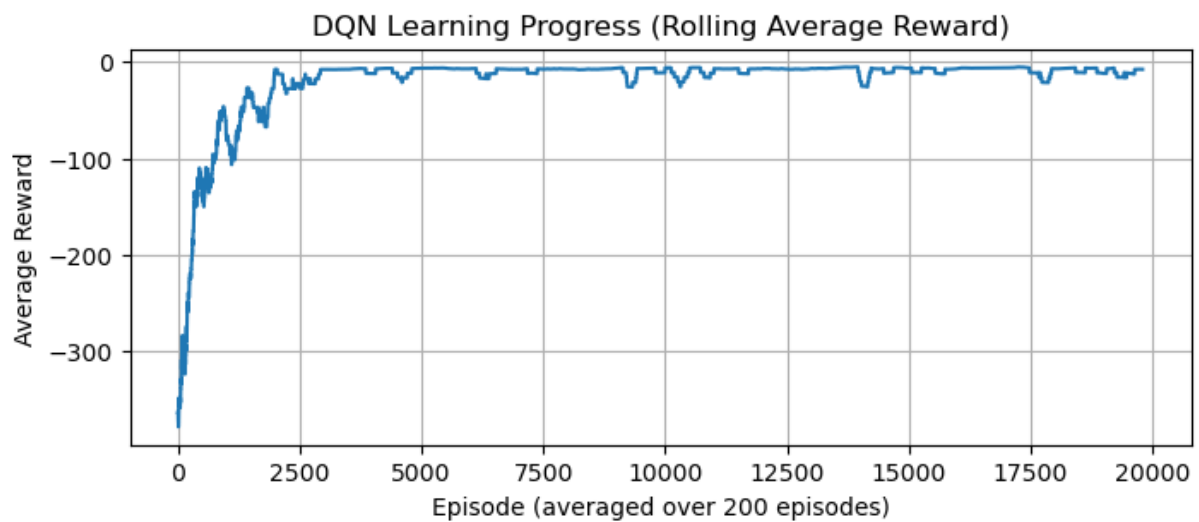


Figure 23: DQN charging schedule and battery trajectory.

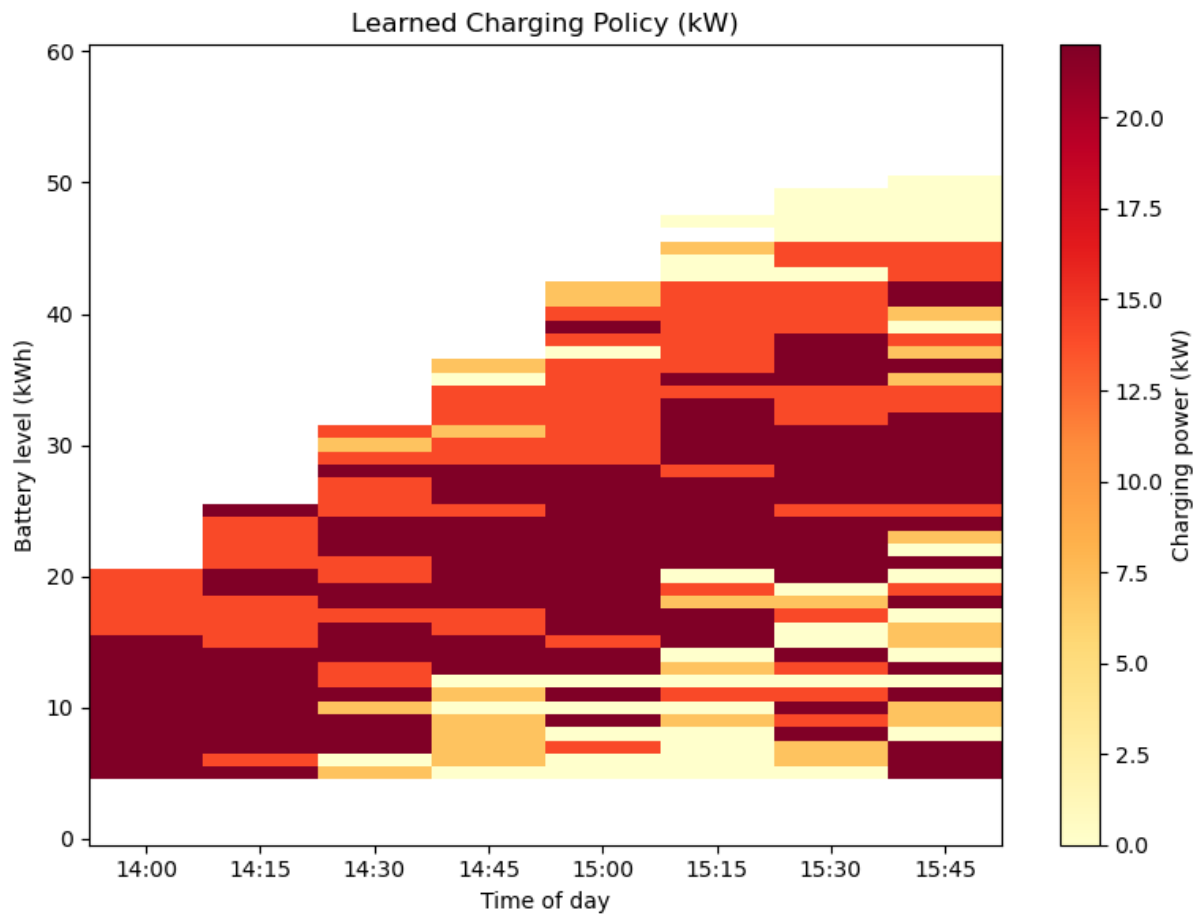


Figure 24: Optimal charging policy across all states.

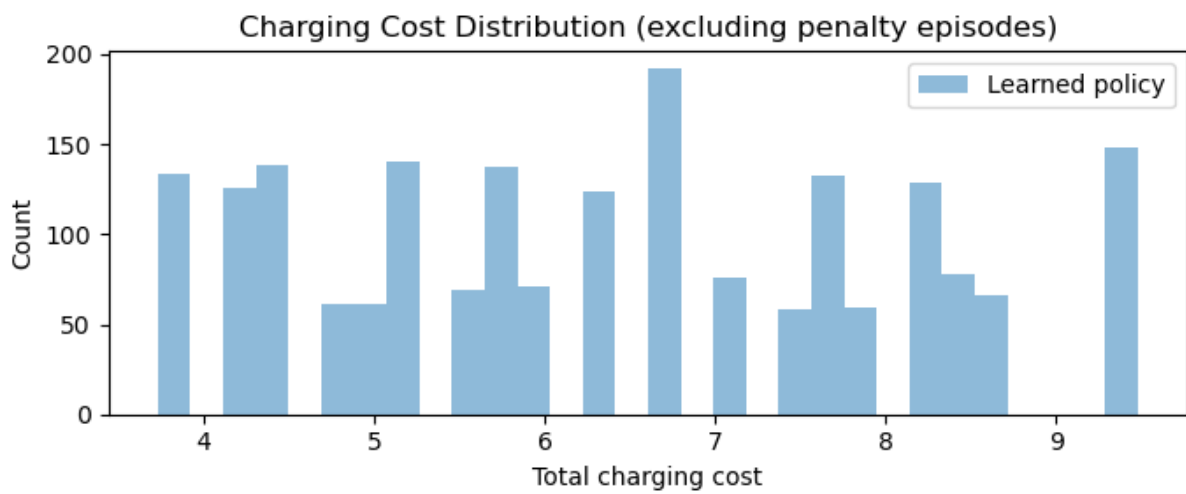


Figure 25: Charging cost distribution under the learned policy.

- City of Chicago. (2024). *Taxi trips (2024-)*. Retrieved July 26, 2026, from <https://data.cityofchicago.org/d/ajtu-isnz>
- City of Chicago. (2025). *Boundaries - community areas*. Retrieved July 26, 2026, from <https://data.cityofchicago.org/Facilities-Geographic-Boundaries/Boundaries-Community-Areas/igwz-8jzy>
- EV Database. (2026). *Useable battery capacity of electric cars*. <https://ev-database.org/cheatsheet/useable-battery-capacity-electric-car>
- Iowa Environmental Mesonet. (n.d.). *Download asos/awos/metar data*. Retrieved July 26, 2026, from <https://mesonet.agron.iastate.edu/request/download.phtml>
- Kaiyi Global. (2026). *Best ev for taxi in 2026*. <https://www.kaiyiglobal.com/blog/best-ev-for-taxi-in-2026-top-5-electric-cars-for-taxi-drivers>
- OpenStreetMap Contributors. (n.d.). *Copyright and license*. Retrieved July 26, 2026, from <https://www.openstreetmap.org/copyright>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Perktold, J., Seabold, S., Sheppard, K., Fulton, C., Shedden, K., et al. (2024, April). *Statsmodels/statsmodels: Release 0.14.2* (Version v0.14.2). Zenodo. <https://doi.org/10.5281/zenodo.10984387>
- Pod Energy. (2026). *How long to charge an electric car*. <https://podenergy.com/guides/how-long-to-charge-an-electric-car>
- python-holidays Contributors. (n.d.). *Python-holidays documentation*. Retrieved July 26, 2026, from <https://holidays.readthedocs.io/en/latest/>
- RAPIDS Development Team. (2023). RAPIDS: Cuml [Accessed: 2026-07-26].
- Raschka, S., Patterson, J., & Nolet, C. (2020). Machine learning in python: Main developments and technology trends in data science, machine learning, and artificial intelligence. *arXiv preprint arXiv:2002.04803*.
- Schwarz, G. (1978). Estimating the dimension of a model. *The Annals of Statistics*, 6(2), 461–464. <https://doi.org/10.1214/aos/1176344136>
- Scikit-learn Developers. (n.d.). *GaussianMixture*. Retrieved July 26, 2026, from <https://scikit-learn.org/stable/modules/generated/sklearn.mixture.GaussianMixture.html>
- Uber Technologies. (n.d.). *H3 documentation*. Retrieved July 26, 2026, from <https://h3geo.org/docs/>
- Warmerdam, V. D., Bruzzesi, F., MBrouns, Collot, S., Kübler, R., de Boer, J., Hoeven, P., Kalimeri, M., Hoogland, K., Masip, D., et al. (2026, July). *Koaning/scikit-lego: 0.9.9* (Version v0.9.9). Zenodo. <https://doi.org/10.5281/zenodo.21210295>